NAME:THOTA LOKESH

REG.NO:192312196

Ex 1: 8-Puzzle Problem (A* Search)

CODE:

```
    import heapq
def manhattan(state, goal):
  dist = 0
  for r in range(3):
    for c in range(3):
      val = state[r][c]
      if val != 0:
        for gr in range(3):
          for gc in range(3):
            if goal[gr][gc] == val:
              dist += abs(r - gr) + abs(c - gc)
  return dist
def solve_8_puzzle(start, goal):
  start_tuple = tuple(tuple(row) for row in start)
  goal_tuple = tuple(tuple(row) for row in goal)
  pq = [(manhattan(start, goal), 0, start_tuple, [])]
  visited = {start_tuple: 0}
  while pq:
    _, g, curr, path = heapq.heappop(pq)
    if curr == goal_tuple:
```

```
            return path + [curr]
        for r in range(3):
            for c in range(3):
                if curr[r][c] == 0:
                    zr, zc = r, c
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = zr + dr, zc + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                nxt = [list(row) for row in curr]
                nxt[zr][zc], nxt[nr][nc] = nxt[nr][nc], nxt[zr][zc]
                nxt_t = tuple(tuple(row) for row in nxt)
                if nxt_t not in visited or g + 1 < visited[nxt_t]:
                    visited[nxt_t] = g + 1
                    heapq.heappush(pq, (g + 1 + manhattan(nxt, goal), g + 1, nxt_t, path + [curr]))
start = [[1, 2, 3], [4, 0, 6], [7, 5, 8]]
goal = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
steps = solve_8_puzzle(start, goal)
print(f"Solved in {len(steps)-1} moves:")
for s in steps:
    for row in s:
        print(row)
    print("---")
```

OUTPUT:

```
============== RESTART: (
Solved in 2 moves:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)
---
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
---
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```